# Enterprise WordPress Performance Operational Checklist

Version 1.1 — DRAFT

Dan Knauss, General Editor

# Enterprise WordPress Performance Operational Checklist

> **Status:** DRAFT **Version:** 1.1 **Date:** 14 June 2026 **General Editor:** Dan Knauss **Currency:** Last verified against WordPress 7.0 on 2026-06-14.

A checklist-oriented enterprise operational guide for diagnosing, optimizing, and verifying performance on high-traffic WordPress sites where backend execution, cacheability, database access, object caching, traffic events, and measurement discipline matter. It complements the general `wordpress-performance-optimization-checklist.md` by going deeper on operational scale and platform constraints.

> Do not start by installing another performance plugin. Start by identifying where time is spent, whether WordPress is executing at all, and whether the response can safely be cached.

---

## Table of Contents

## 0. Define the performance problem

- [ ] Identify the affected surface:
    - [ ] public anonymous page
    - [ ] logged-in page
    - [ ] editorial/admin workflow
    - [ ] API endpoint
    - [ ] AJAX endpoint
    - [ ] search results
    - [ ] sitemap generation
    - [ ] redirect path
    - [ ] high-traffic event page
- [ ] Identify the failure mode:
    - [ ] high TTFB
    - [ ] slow PHP execution
    - [ ] slow MySQL queries
    - [ ] too many database queries
    - [ ] cache misses
    - [ ] object-cache pressure
    - [ ] cache stampede/race condition
    - [ ] slow third-party/external request
    - [ ] client-side JavaScript delay
    - [ ] load-test failure

- [ ] Confirm the test target URL or route.
- [ ] Record whether the request is anonymous or authenticated.
- [ ] Record whether the page should be cacheable.
- [ ] Confirm rollback path before changing production behavior.

---

## 1. Trace the request cycle

For each target request, walk the full path.

- [ ] User enters URL.
- [ ] DNS resolves the hostname.
- [ ] Browser sends HTTP request.
- [ ] Edge/CDN checks for cached response.
- [ ] Origin web server receives request if edge misses or bypasses.
- [ ] WordPress core loads.
- [ ] MU plugins load.
- [ ] Plugins and theme code run.
- [ ] Database and object cache are queried.
- [ ] Template is selected and rendered.
- [ ] HTTP response is sent to browser.
- [ ] Browser parses HTML, downloads assets, executes JavaScript, renders the page.
- [ ] Additional requests fire: AJAX, REST, media, tracking, personalization.

Decision point:

- [ ] If the edge cache returns a valid `HIT`, focus on payload, browser rendering, and post-load requests.
- [ ] If the request reaches origin, focus on cacheability, PHP, database, object cache, hooks, templates, and external calls.

---

## 2. Establish a baseline

Run the same measurements before and after every change.

- [ ] Measure uncached TTFB.
- [ ] Measure warm-cache TTFB.
- [ ] Capture cache headers.
- [ ] Capture response status and redirects.

- [ ] Capture PHP/application timing.
- [ ] Capture database query count and slow queries.
- [ ] Capture object-cache behavior when available.
- [ ] Record WordPress version, PHP runtime, and database version. WordPress 7.0 requires PHP 7.4.0 or higher; PHP 8.3 remains the recommended baseline for modern performance work.
- [ ] Capture frontend impact if browser rendering is part of the symptom.
- [ ] Identify AI Client, Abilities API, Connectors API, or other external-service integrations that can add latency, retries, or queue pressure.

Useful command:

```
curl -s -o /dev/null -D headers.txt \
  -w "dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} ttfb:%
   https://example.com/target/
```

Baseline table:

| Target | User state | Expected cacheability | Cache status | TTFB | PHP time | DB time | Notes |
|---|---|---|---|---|---|---|---|
| `/` | anonymous | cacheable | | | | | |
| `/search` | anonymc | maybe dy-namic | | | | | |
| `/wp-json/...` | varies | endpoint-specific | | | | | |

---

### 3. DNS and HTTP request setup

- [ ] Confirm DNS resolves quickly and predictably.
- [ ] Avoid unnecessary DNS indirection and stale records.
- [ ] Confirm final canonical URL does not require multiple redirects.
- [ ] Confirm HTTPS works without certificate or protocol negotiation issues.
- [ ] Confirm HTTP/2 or HTTP/3 support where appropriate.
- [ ] Check request headers that may vary cache behavior:
  - [ ] cookies

    - [ ] authorization
    - [ ] query strings
    - [ ] user-agent variation
    - [ ] language/country headers

Redirect check:

```
curl -IL https://example.com/some-url/
```

- [ ] If multiple redirects occur, identify the source: WordPress, plugin, CDN, web server, localization, or application code.
- [ ] Look for `x-redirect-by` when WordPress generates redirects.

Examples seen in the course:

```
x-redirect-by: WordPress
x-redirect-by: Polylang Pro
```

Debug filter example:

```
add_filter( 'x_redirect_by', function( $x_redirect_by, $status, $location ) {
    wp_die( var_dump( array(
        $x_redirect_by,
        $status,
        $location,
        wp_debug_backtrace_summary(),
    ) ) );
}, 10, 3 );
```

Use that only in safe debugging contexts, not as a production-facing change.

---

### 4. Full-page cache and edge cache

Full-page caching is the first major enterprise performance boundary: if the edge can serve HTML, WordPress and MySQL may not need to run.

- [ ] Identify pages that should be edge-cacheable.
- [ ] Identify pages that must bypass cache:
    - [ ] logged-in personalized views
    - [ ] carts

  - [ ] checkout
  - [ ] account pages
  - [ ] previews
  - [ ] nonce-sensitive forms
  - [ ] private API responses

- [ ] Check cache status response headers.
- [ ] Confirm `Cache-Control` behavior is intentional.
- [ ] Confirm cookies are not causing broad cache bypasses.
- [ ] Confirm query parameters are not fragmenting the cache key unnecessarily.
- [ ] Confirm TTL is appropriate for content volatility.
- [ ] Confirm invalidation/purge logic is precise enough.
- [ ] Investigate cache misses before tuning PHP.

Edge-cache triage:

- [ ] Is the response cacheable?
- [ ] Does it include `Set-Cookie`?
- [ ] Does it vary by cookie or authorization?
- [ ] Is the TTL too short?
- [ ] Is a plugin sending no-cache headers?
- [ ] Is the request URL unique because of query strings?
- [ ] Are purges too broad or too frequent?

---

### 5. TTL, headers, and invalidation

- [ ] Set long TTLs for stable public content.
- [ ] Set shorter TTLs for content that changes frequently.
- [ ] Use purge/invalidation for freshness rather than universally short TTLs.
- [ ] Avoid sending no-cache/no-store on pages that should be public-cacheable.
- [ ] Verify generated headers from WordPress, plugins, CDN, and server rules.
- [ ] Ensure invalidation happens when content changes.
- [ ] Avoid purging the whole site for small content updates when targeted purges are possible.

Header review:

```
curl -I https://example.com/page/
```

Look for:

- [ ] `Cache-Control`
- [ ] `Expires`
- [ ] `Vary`
- [ ] `Set-Cookie`
- [ ] CDN cache status headers
- [ ] custom platform cache headers

---

## 6. Application optimization

If a request reaches WordPress, optimize the application path.

- [ ] Identify expensive hooks, filters, and actions.
- [ ] Confirm callbacks run only where needed.
- [ ] Avoid global work during every request.
- [ ] Avoid excessive template complexity.
- [ ] Avoid template over-use where many partials perform redundant work.
- [ ] Cache expensive partial output when safe.
- [ ] Eliminate uncached functions on hot paths.
- [ ] Avoid remote HTTP calls during page rendering.
- [ ] Move non-critical work to async/background processing.

Hook review checklist:

- [ ] Does this callback run on frontend, admin, REST, AJAX, or all contexts?
- [ ] Does it execute database queries?
- [ ] Does it call external services?
- [ ] Does it build data that could be cached?
- [ ] Does it run for users/pages that do not need it?

---

## 7. Partial output caching

Use partial output caching for expensive fragments that cannot be solved by full-page cache alone. See `DEVELOPER_REFERENCE.md` §18 for the canonical cache-key safety checklist.

**Cache-key safety checklist for fragment / partial-output caching**

Before caching a fragment with `wp_cache_set()` or `set_transient()`, verify that the key includes every dimension that varies the output:

- **Site / blog scope:** `get_current_blog_id()` on multisite.
- **Locale:** `get_locale()` or `determine_locale()` if locale affects content.
- **Currency / region:** any active currency, region, or store context.
- **Auth state:** logged-in vs logged-out as separate keys.
- **User identity:** only when the fragment is intentionally per-user and you have evaluated privacy and cardinality implications.
- **Role / capability:** if the fragment differs by role or capability, include the relevant subset.
- **A/B / personalization bucket:** include the bucket identifier.
- **Query parameters that change output:** explicit allowlist, never raw `$_GET`.

Privacy and cardinality:

- **Never** cache personalized output under a shared key. One shared-key mistake can leak one user's content to another.
- Per-user keys grow cache footprint linearly with active users. Bound this with short TTLs and use them only when regenerate cost dominates.
- On persistent object caches, monitor key cardinality and memory pressure for any per-user fragment cache group.

Invalidation triggers:

- Source content change: `save_post`, custom post type hooks, ACF/CMB2 update hooks.
- User profile / role / capability change: `set_user_role`, `profile_update`.
- Locale / settings change.
- Multisite blog switch / domain change.
- [ ] Identify repeated expensive fragments: sidebars, related posts, navigation, widgets, API-derived blocks.
- [ ] Create cache keys that include only the context that changes output.
- [ ] Set an expiration appropriate to freshness needs.
- [ ] Invalidate when source data changes where necessary.
- [ ] Avoid caching personalized fragments under shared keys.

Example pattern:

```
<?php
$role_fragment = is_user_logged_in() ? 'logged_in' : 'logged_out';
$cache_key     = sprintf(
    'recent_posts_sidebar_v2_blog%d_locale%s_state%s',
    get_current_blog_id(),
```

```
    sanitize_key( determine_locale() ),
    $role_fragment
);

$recent_posts_html = wp_cache_get( $cache_key, 'page_fragments' );

if ( false === $recent_posts_html ) {
    ob_start();
    // Generate the expensive sidebar output here.
    get_template_part( 'partials/recent-posts-sidebar' );
    $recent_posts_html = ob_get_clean();

   wp_cache_set( $cache_key, $recent_posts_html, 'page_fragments', HOUR_IN_SECONDS
}

echo $recent_posts_html;
```

---

### 8. MySQL query optimization

Slow database work is one of the most common origin-side bottlenecks.

- [ ] Count total queries for the request.
- [ ] Identify slow queries.
- [ ] Identify duplicate queries.
- [ ] Identify unbounded queries.
- [ ] Identify queries that scan too many rows.
- [ ] Avoid expensive meta queries on high-cardinality data.
- [ ] Avoid taxonomy over-use for query patterns that do not scale.
- [ ] Avoid `NOT IN` when it forces inefficient exclusion scans.
- [ ] Avoid leading-wildcard `LIKE` searches on large tables.
- [ ] Use search infrastructure such as Elasticsearch/OpenSearch for heavy search workloads.
- [ ] Use `EXPLAIN` to understand execution plans.

WP_Query baseline example:

```
$query = new WP_Query(
    array(
        'post_type'      => 'post',
        'posts_per_page' => 5,
```

```
        'orderby'        => 'date',
        'order'          => 'DESC',
    )
);
```

Query anti-pattern checks:

- [ ] `posts_per_page => -1` on large datasets.
- [ ] missing pagination.
- [ ] meta queries used as primary filters at scale.
- [ ] taxonomy queries stacked into complex intersections/exclusions.
- [ ] `post__not_in` or SQL `NOT IN` against large sets.
- [ ] `LIKE '%term%'` on large text columns.
- [ ] ordering by unindexed or computed values.

---

### 9. Unbounded queries

Unbounded queries are dangerous because they look safe during development and fail at enterprise scale.

- [ ] Search code for `posts_per_page => -1`.
- [ ] Search code for `numberposts => -1`.
- [ ] Search code for falsy IDs accidentally becoming broad queries.
- [ ] Add explicit limits.
- [ ] Paginate large result sets.
- [ ] Use caching for expensive repeated query results.
- [ ] Validate IDs and input before building queries.

Examples:

```
$post_id = intval( $my_unexpectedly_falsy_post_id );
$query_args = array(
    'p'              => $post_id,
    'posts_per_page' => -1,
);
$args = array(
    'post_type' => 'post',
    'date_query' => array(
        'after'  => '2021-01-01',
        'before' => '2021-12-31',
```

```
    ),
);

$query = new WP_Query( $args );
```

Safer pattern:

```
function limit_news_scope_query_filter( $query ) {
    if ( is_admin() || ! $query->is_main_query() ) {
        return;
    }

    if ( $query->is_category( 6 ) ) {
        $query->set( 'posts_per_page', 20 );
    }
}
add_action( 'pre_get_posts', 'limit_news_scope_query_filter' );
```

---

### 10. Taxonomies, exclusions, post meta, and LIKE

#### Taxonomies

- [ ] Use taxonomies for classification, not arbitrary high-cardinality data.
- [ ] Avoid overly complex taxonomy intersections/exclusions.
- [ ] Keep term relationships manageable.
- [ ] Use dedicated indexes/search systems for large faceted search use cases.

Example scope limiter:

```
function limit_news_scope_query_filter( $query ) {
    if ( is_admin() || ! $query->is_main_query() ) {
        return;
    }

    if ( $query->is_category() ) {
        $query->set( 'posts_per_page', 20 );
    }
}
add_action( 'pre_get_posts', 'limit_news_scope_query_filter' );
```

**NOT IN**

- [ ] Avoid large `NOT IN` exclusions.
- [ ] Prefer positive selection rules where possible.
- [ ] Cache exclusion sets if they must be computed.
- [ ] Test with production-scale data.

Example to scrutinize:

```
SELECT * FROM wp_posts
WHERE post_type = 'post'
  AND post_status = 'publish'
  AND ID NOT IN (1, 2, 3);
```

**Post meta**

- [ ] Do not treat post meta as a general-purpose relational store at scale.
- [ ] Avoid sorting/filtering large result sets by arbitrary meta values.
- [ ] Consider custom tables or search infrastructure for high-volume structured data.

**LIKE**

- [ ] Avoid leading wildcard searches on large columns.
- [ ] Prefer indexed prefix searches if acceptable.
- [ ] Use search infrastructure for real search workloads.

Examples:

```
SELECT * FROM wp_posts WHERE post_title LIKE '%WordPress%';
SELECT * FROM wp_posts WHERE post_title LIKE 'WordPress%';
```

The second pattern can be more index-friendly than the first, but still needs production-scale testing.

---

## 11. EXPLAIN-driven query review

Use `EXPLAIN` to verify how MySQL plans to execute a query.

- [ ] Run `EXPLAIN` on slow queries.

- [ ] Check which table is scanned.
- [ ] Check access type.
- [ ] Check possible keys.
- [ ] Check the key actually used.
- [ ] Check estimated rows scanned.
- [ ] Look for filesort or temporary table usage.
- [ ] Compare plans before and after query/index changes.

Examples:

```
EXPLAIN SELECT * FROM wp_posts WHERE post_date > '2023-01-01';
EXPLAIN
SELECT wp_posts.*, wp_postmeta.meta_value
FROM wp_posts
JOIN wp_postmeta ON wp_posts.ID = wp_postmeta.post_id
WHERE wp_posts.post_date > '2023-01-01';
```

---

## 12. Object caching

### Cache-layer terminology

Three distinct WordPress caching subsystems are often conflated. `WP_CACHE` is a constant that allows WordPress to load the `wp-content/advanced-cache.php` drop-in for **page caching** (saving complete rendered HTML). `wp-content/object-cache.php` is a separate drop-in for **persistent object caching** (Redis/Memcached, caching database/query/application objects across requests). The Transients API uses object cache when persistent caching is configured, otherwise falls back to the `wp_options` table. See `REFERENCE-WP-Transients-Persistent-Object-Cache.md` and `DEVELOPER_REFERENCE.md` §4 for full treatment.

Object caching reduces repeated computation and repeated database access.

- [ ] Confirm a persistent object cache exists when the site needs one.
- [ ] Confirm cache service health.
- [ ] Track hit rate and miss rate.
- [ ] Cache expensive query results and computed values.
- [ ] Use cache groups intentionally.
- [ ] Keep keys stable and specific.
- [ ] Avoid excessive cache calls in tight loops.
- [ ] Avoid storing huge objects that increase memory pressure.

- [ ] Include `alloptions` and autoloaded-option size in object-cache review. WordPress 6.6+ uses on, `off`, `auto`, `auto-on`, and `auto-off` values while upgraded rows may still use `yes`/no; monitoring must query both vocabularies.
- [ ] Avoid caching personalized/private data with shared keys.

---

## 13. Transients and in-memory caching

Transients:

- [ ] Use transients for data that can expire.
- [ ] Set realistic expiration times.
- [ ] Account for either database-backed or object-cache-backed storage.
- [ ] Avoid relying on transients for data that must always exist.
- [ ] Clean up transient buildup when needed.

In-memory request-level caching:

- [ ] Use static variables for repeat computations within one request.
- [ ] Do not use request-local cache as a substitute for persistent cache across requests.

Example:

```
function getExpensiveComputationResult( $input ) {
    static $cache = array();

    if ( isset( $cache[ $input ] ) ) {
        return $cache[ $input ];
    }

    $result = perform_expensive_computation( $input );
    $cache[ $input ] = $result;

    return $result;
}
```

---

## 14. Race conditions and cache stampedes

Race conditions occur when multiple requests try to rebuild the same cache item at once. Cache stampedes happen when many requests miss simultaneously and all perform the expensive work.

- [ ] Add locking around expensive cache regeneration.
- [ ] Use soft TTL/hard TTL patterns where appropriate.
- [ ] Refresh important cache entries before they expire during high traffic.
- [ ] Avoid synchronized expiration for many hot keys.
- [ ] Add jitter to expirations where useful.
- [ ] Serve stale data briefly when freshness requirements allow it.

Lock example:

These lock patterns assume a **persistent object cache** with atomic add semantics, such as Redis or Memcached via `wp-content/object-cache.php`. On default WordPress with no persistent object cache, `wp_cache_add()` is request-local: every concurrent request can "acquire" the lock simultaneously, so it provides no cross-request coordination. Confirm a persistent object cache is in place before relying on this pattern.

Fallbacks when persistent object caching is not available:

- Use a database-backed mutex or a short-lived transient/option lock, accepting the database-write cost and failure modes.
- Use a platform-provided distributed lock service if the host offers one.
- Use stale-while-revalidate: serve the previous value while a warmer regenerates asynchronously.
- Pre-warm hot keys outside user requests with cron, deploy hooks, or scheduled actions.
- Restructure so the expensive operation never runs inside a user request.

```
$lock_key = 'my_cache_lock';
$lock_acquired = wp_cache_add( $lock_key, true, 'locks', 30 );

if ( $lock_acquired ) {
    $data = regenerate_expensive_cache_value();
   wp_cache_set( 'my_cache_key', $data, 'my_group', HOUR_IN_SECONDS );
    wp_cache_delete( $lock_key, 'locks' );
} else {
    $data = wp_cache_get( 'my_cache_key', 'my_group' );
}
```

---

### 15. WP-Cron and scheduled work

WP-Cron can create request-time spikes if scheduled work runs during user traffic.

- [ ] Identify due cron events.

- [ ] Identify slow recurring jobs.
- [ ] Run cron through real server cron where appropriate.
- [ ] Disable request-triggered WP-Cron only after the external runner is verified.
- [ ] Avoid heavy jobs during peak traffic.
- [ ] Batch large jobs.
- [ ] Ensure jobs are idempotent.
- [ ] Monitor failures and backlog.

**Sequence matters.** Install and verify the external runner before setting `DISABLE_WP_CRON`. If the constant is set first, scheduled events and Action Scheduler queues can stop until the runner works.

WP-CLI cron runner:

```
* * * * * cd /path/to/site && wp cron event run --due-now --quiet
```

Verify before changing `wp-config.php`:

```
wp cron event list --due-now
# Check scheduler logs and application logs for successful invocations.
```

Only after verification:

```
define( 'DISABLE_WP_CRON', true );
```

Fallback PHP runner:

```
* * * * * php /path/to/wordpress/wp-cron.php > /dev/null 2>&1
```

HTTP fallback:

```
* * * * * curl -s https://example.com/wp-cron.php?doing_wp_cron >/dev/null 2>&1
```

After setting the constant, confirm scheduled events continue to clear. WordPress 6.9+ spawns request-triggered WP-Cron at shutdown, which can reduce user-facing latency, but enterprise/high-traffic sites should still use a verified external runner. Roll back by commenting out the constant and clearing OPcache if the backlog grows.

## 16. AJAX, REST, sitemaps, redirects, and external requests

### AJAX and REST

- [ ] Identify `admin-ajax.php` calls on public pages.
- [ ] Identify REST API calls after initial render.
- [ ] Remove unnecessary polling.
- [ ] Cache public endpoint responses where safe.
- [ ] Move heavy endpoint work out of synchronous user requests.
- [ ] Authenticate private endpoints correctly.

### Sitemaps

- [ ] Ensure sitemap generation does not perform expensive unbounded queries.
- [ ] Cache sitemap output when possible.
- [ ] Paginate large sitemaps.
- [ ] Avoid regenerating large sitemap structures on every request.

### External requests

- [ ] Identify remote HTTP calls during page generation.
- [ ] Add strict timeouts.
- [ ] Cache remote responses.
- [ ] Fail gracefully when the remote service is slow.
- [ ] Move non-critical requests to background processing.

---

## 17. Client-side JavaScript and browser rendering

Even when backend performance is good, the browser can still feel slow.

- [ ] Identify heavy frontend scripts.
- [ ] Remove duplicate libraries.
- [ ] Avoid loading plugin scripts globally when only one page needs them.
- [ ] Defer non-critical JavaScript.
- [ ] Reduce long main-thread tasks.
- [ ] Measure interaction delay after script changes.
- [ ] Watch for AJAX calls triggered by frontend scripts after load.
- [ ] Verify WordPress 6.3+ `fetchpriority` handling before adding platform-level hero-image preloads.

- [ ] Review WordPress 6.8+ speculative loading rules alongside analytics, consent, personalization, and cacheability controls.
- [ ] Re-test authenticated admin/editor workflows after WordPress 7.0 upgrades, especially custom blocks, editor extensions, Font Library workflows, Visual Revisions, and iframed-editor behavior.

---

## 18. Traffic patterns and high-traffic events

High-traffic events require preparation, not just reaction.

- [ ] Identify expected traffic sources and timing.
- [ ] Estimate peak requests per minute/second.
- [ ] Identify pages that will receive traffic.
- [ ] Pre-warm caches.
- [ ] Freeze risky deployments before the event.
- [ ] Audit cacheability of campaign landing pages.
- [ ] Remove avoidable query-string cache fragmentation.
- [ ] Exclude authenticated, cart/checkout, preview, session-keyed, and personalized paths from speculative loading before enabling more aggressive prefetch/prerender behavior.
- [ ] Confirm origin can survive expected cache misses.
- [ ] Confirm monitoring and alerting are active.
- [ ] Prepare rollback and incident contacts.

High-traffic preflight:

- [ ] CDN/edge cache hit ratio acceptable.
- [ ] Origin TTFB acceptable under warm cache.
- [ ] No heavy uncached widgets on landing page.
- [ ] No slow external dependency in critical path.
- [ ] No scheduled jobs during peak window.
- [ ] Load test completed in a safe environment.

---

## 19. Load testing

Load testing should be coordinated and scoped; never surprise production.

- [ ] Get explicit approval.

- [ ] Define target URLs and traffic model.
- [ ] Define acceptable limits and stop conditions.
- [ ] Test cacheable and uncached paths separately.
- [ ] Monitor origin, database, object cache, and CDN during the test.
- [ ] Compare behavior to expected real traffic.

Example:

```
k6 run --vus 100 --duration 5m script.js
```

Record:

- [ ] requests per second
- [ ] p50/p95/p99 latency
- [ ] error rate
- [ ] cache hit ratio
- [ ] PHP/application time
- [ ] database pressure
- [ ] object-cache pressure

---

## 20. Measuring time spent

Use multiple tools because each sees a different layer.

### PHP logging

- [ ] Use targeted timing around suspected code paths.
- [ ] Remove noisy debug logging after investigation.
- [ ] Do not expose timing details publicly.

Example:

```
function my_custom_function() {
    $start_time = microtime( true );

    // Code to be measured here.

    $end_time = microtime( true );
    $execution_time = $end_time - $start_time;

   error_log( 'my_custom_function took ' . $execution_time . ' seconds' );
}
```

**New Relic**

- [ ] Use APM traces to find slow transactions.
- [ ] Group by transaction type: page, admin, REST, AJAX, cron.
- [ ] Inspect database, external service, and PHP segment timing.
- [ ] For WordPress 7.0 AI/connectors or similar integrations, separate external API latency, retries, timeouts, queueing, credential access failures, and cacheability of generated results.

Example:

```
// Guard so the code is safe whether the New Relic PHP agent is loaded or not.
if ( extension_loaded( 'newrelic' ) && function_exists( 'newrelic_name_transaction'
    // Name the current transaction so it appears as a distinct entry
    // in the New Relic APM transaction list. Avoid per-user or per-
URL names.
    newrelic_name_transaction( 'myplugin/expensive-path' );
}

// Optional: add custom attributes for filtering and querying in NRQL.
if ( function_exists( 'newrelic_add_custom_parameter' ) ) {
   newrelic_add_custom_parameter( 'plugin_context', 'myplugin_expensive_path' );
}
```

Use `newrelic_name_transaction()` to give the current request a stable, low-cardinality name in APM. The New Relic transaction-start API takes a New Relic application name, not a transaction name, and is reserved for advanced transaction-lifecycle patterns such as queue workers manually ending one transaction and starting another.

**Query Monitor**

- [ ] Inspect query count and slow queries.
- [ ] Inspect hooks and component timing.
- [ ] Inspect HTTP API calls.
- [ ] Use custom timers for targeted sections.

Example:

```
do_action( 'qm/start', 'mytimer' );

foreach ( range( 1, 10 ) as $i ) {
```

```
    function_to_measure_time_spent( $i );
}

do_action( 'qm/stop', 'mytimer' );
```

---

## 21. Performance alerting

- [ ] Alert on high TTFB.
- [ ] Alert on elevated error rate.
- [ ] Alert on database saturation or slow queries.
- [ ] Alert on object-cache service failures.
- [ ] Alert on CDN cache hit ratio drops.
- [ ] Alert on queue/cron backlogs.
- [ ] Alert on external dependency latency.
- [ ] Alert on sudden traffic spikes.

Good alerts should include:

- [ ] affected URL or transaction
- [ ] severity
- [ ] current value vs threshold
- [ ] duration
- [ ] likely owner
- [ ] first diagnostic link
- [ ] rollback/escalation instructions

---

## 22. Verification and definition of done

After each optimization:

- [ ] Re-run the same measurement from the baseline.
- [ ] Confirm the same user state and URL were tested.
- [ ] Confirm cache status is expected.
- [ ] Confirm no private data is cached publicly.
- [ ] Confirm no critical functionality regressed.
- [ ] Confirm no new PHP errors.
- [ ] Confirm query count/time improved if database was targeted.
- [ ] Confirm cache hit ratio improved if caching was targeted.

- [ ] Document before/after values.
- [ ] Document remaining bottlenecks.

Done means:

- [ ] The bottleneck has been identified, not guessed.
- [ ] The fix addresses that bottleneck directly.
- [ ] The improvement is measurable.
- [ ] The change is safe under expected traffic.
- [ ] Monitoring can detect regression.

---

## References


- Options API: Disabling autoload for large options (Make WP Core)
- New option functions in 6.4 (Make WP Core)
- WordPress 6.6 Performance Improvements
- Speculative Loading in 6.8
- WordPress 6.8 Performance Improvements
- WordPress 6.9 Frontend Performance Field Guide
- WordPress 6.9 Field Guide
- WordPress 7.0 Field Guide
- Dropping support for PHP 7.2 and 7.3
- Real-time collaboration will not ship in WordPress 7.0
- WordPress release archive
- `wp_set_option_autoload()`
- Performance Lab plugin
- web.dev — Interaction to Next Paint
- Introducing INP to Core Web Vitals (Google Search Central)


### Source note

One WP VIP Learn WP-Cron lesson shows the cron URL as `http://yourdomain.com/wp-cron.php?doing_wp_cron`. That is insecure for any modern production site — credentials, cookies, and the WP-Cron token would travel in cleartext. Use `https://` matching the site's canonical scheme. This operational checklist at §15 already uses HTTPS.